

TD5 : Dénivelés de randonnées

Ce sujet est une synthèse des travaux avec OpenRouteService et avec Json.

Le fichier `mini-rando_gps.json` fournit des séries de positions GPS correspondant à des traces GPS de sorties randonnée de M. Toulemonde. On cherchera dans ce TD à écrire un programme calculant les dénivelés cumulés positif et négatif de chacune de ces randonnées. Pour cela, vous utiliserez l'outil Elevataion de l'API OpenRouteService. On rappelle que la documentation de OpenRouteService est disponible à cette adresse : https://rtavenar.github.io/doc_openrouteservice/.

Préambule

- Dans votre dossier `PythonOpenData`, créez un sous-répertoire `TD5` dans lequel vous réaliserez l'ensemble des travaux.
- Lancer l'éditeur Visual Studio Code et ouvrir le dossier `PythonOpenData/TD5`.
- Télécharger le fichier `mini-rando_gps.json` dans le dossier `PythonOpenData/TD5`.
- Télécharger le fichier source `td5.py` dans lequel vous écrirez le code du TD.
- Copier le fichier `credentials.json` du TD3 dans le dossier `PythonOpenData/TD5`.

Exercice 1 : Altitudes avec OpenRouteService

- Q.1.1. Écrire une fonction `altitude_coord` qui prend en entrée une clé d'API, les coordonnées d'un lieu et retourne l'altitude de ce lieu.
- Q.1.2. En utilisant la fonction précédente, écrire une fonction `altitude_adresse` qui prend en entrée une clé d'API, l'adresse d'un lieu, et retourne l'altitude de ce lieu. Tester en affichant l'altitude de Rennes, de Saint-Malo puis de Chamonix.
- Q.1.3. Ecrire une fonction `altitudes_lst_coords` qui prend en entrée une clé d'API, une liste de positions GPS (chacune codée sous la forme d'un dictionnaire comme précisé plus bas) et retourne une liste d'altitudes. Vous pourrez utiliser l'exemple suivant pour vos tests :

```
lst_gps = [  
    {"lng": -1.426533, "lat": 48.005135},  
    {"lng": -1.418127, "lat": 47.986058},  
    {"lng": -1.427611, "lat": 47.989871},  
    {"lng": -1.430202, "lat": 48.000354}  
]
```

Exercice 2 : Dénivelés

En utilisant la fonction écrite à la question précédente, écrire une fonction qui prend en entrée une liste de positions GPS et une clé d'API et retourne la somme des dénivelés positifs (d'une part) et négatifs (d'autre part), entre les points GPS successifs.

Par exemple, en appelant cette fonction avec la liste de de coordonnées GPS précédente, on devrait obtenir la paire de dénivelé (29.07, 28.55). En effet, les altitudes des points GPS étant les suivantes : [36, 68, 49, 35], la somme des dénivelés positif et négatifs donne (32, 33).

Exercice 3 : Analyse de randonnées

En utilisant la fonction écrite à la question précédente, écrire une fonction qui prend en entrée un nom de fichier JSON (contenant des informations sur diverses randonnées) et une clé d'API et affiche, pour chaque randonnée, son nom (attribut "name") et la somme de ses dénivelés positifs (d'une part) et négatifs (d'autre part).

Pour le fichier mini-rando-gps.json, on doit obtenir une sortie du type :

```
TraceGPS Le lngg de la quincampoix - Pire-sur-Seiche
  - Dénivelé positif cumulé : 14
  - Dénivelé négatif cumulé : 14
TraceGPS Issued Messac - CIRCUIT DU PORT
  - Dénivelé positif cumulé : 13
  - Dénivelé négatif cumulé : 13
TraceGPS Issued Coemes-Retiers
  - Dénivelé positif cumulé : 75
  - Dénivelé négatif cumulé : 75
```

Exercice 4 : Fichier de synthèse des randonnées

Écrire une fonction qui prend en entrée un nom de fichier JSON (contenant des informations sur diverses randonnées) et une clé d'API, et écrit dans un nouveau fichier JSON (dont le nom sera passé en paramètre de la fonction) une liste de dictionnaires contenant le nom de la randonnée et les informations de dénivelés positif et négatif, soit quelque chose du type :

```
[
  {
    "name": "TraceGPS Le lngg de la quincampoix - Pire-sur-Seiche",
    "D+": 14,
    "D-": 14
  },
  {
    "name": "TraceGPS Issued Messac - CIRCUIT DU PORT",
    "D+": 13,
    "D-": 13
  },
  {
    "name": "TraceGPS Issued Coemes-Retiers ",
    "D+": 75,
    "D-": 75
  }
]
```